Project Guideline

Godlov Tabow Manfouo

Mert Geceli

The following documentation is the guideline how to set up the project. Normally, everything should be set up like in the following guideline. But to be sure follow the instructions. If something is missing do it like explained.

ATTENTION!!! Before you start with anything plug in the orange ethernet cable to the PC and start the PC after. Only this way is the camera available. Now you can start with the project.

The password for the Virtual Machine: laiT8iet

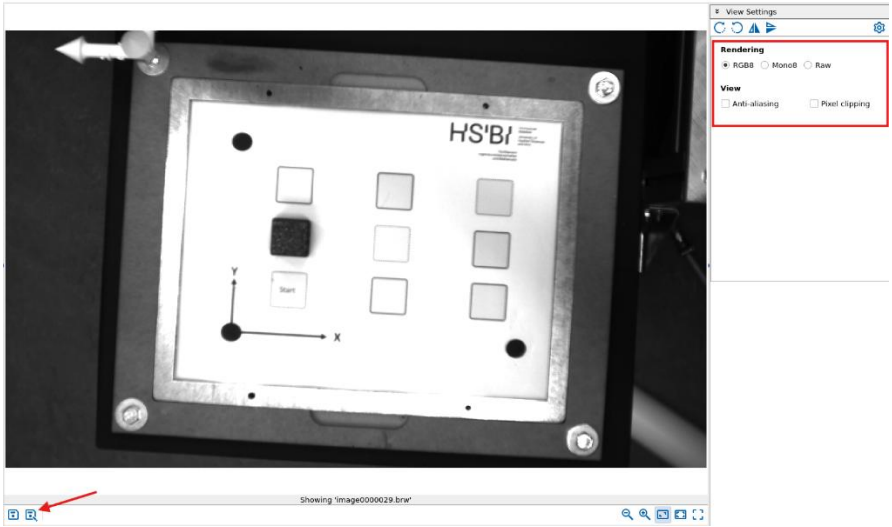
First, you need the relevant codes, which can be found in the Virtual Machine under this path: home/kukaVision/kukavision/KukaVision/repo/KukaVision/KUKA_WS2526/Blob_Detection. Or you can download them from ILIAS and open them on CodeBlocks in the Virtual Machine. You need following codes:

1. cubeDetection
2. circleDetection
3. CamaraConnCubeDet

CamaraConnCubeDet is the main code. This runs the program and moves the robot. The other two codes are for supportive means. **CubeDetection** is to check, whether the camera is detecting the cube on the workplace. **CircleDetection** is for the calibration.

Before we do anything with the codes, open the camera application on the Virtual Machine. We need for the project two images of the workplace. One with and one without the cube. In the application the camera should be seen on the interface. Select it, take the two images and save them like in the following pictures.

The images with the cube are to be named as “cubeT1” and the image without the cube “reference.” The image must be turned 180°.

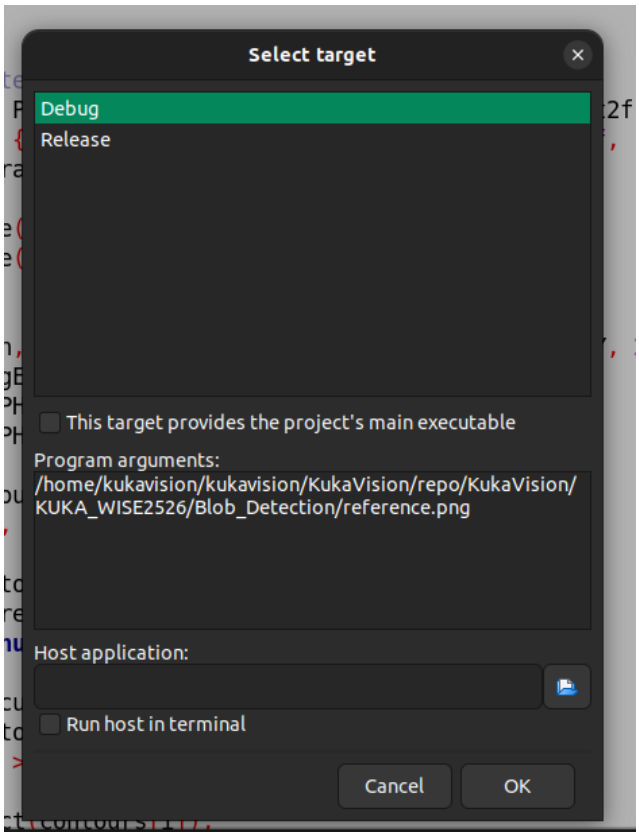


Please close the application after taking the images. Otherwise, the main code does not work because the camera is used by the application.

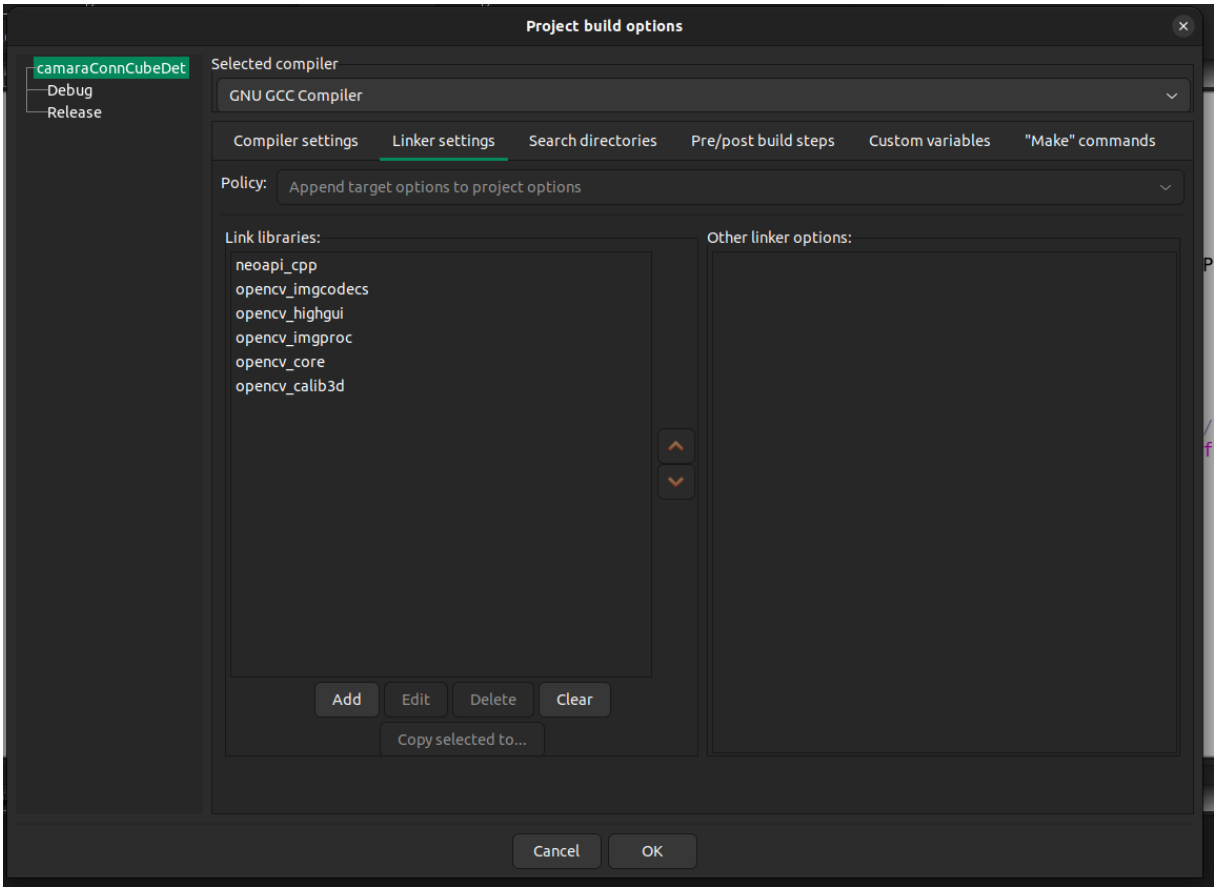
Now, you must adjust some settings in CodeBlocks for each Code. The adjustments are identical.

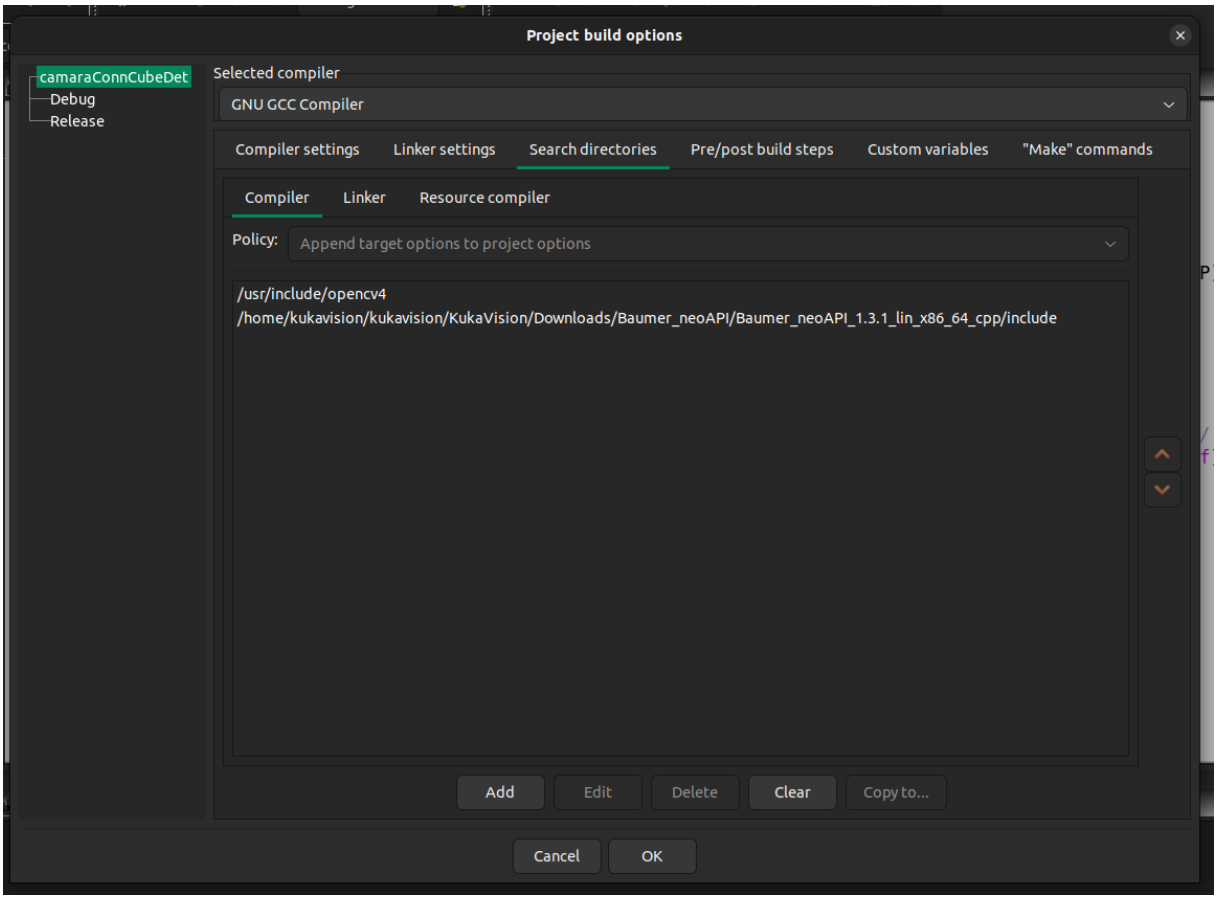
<u>camaraConnCubeDet</u>:

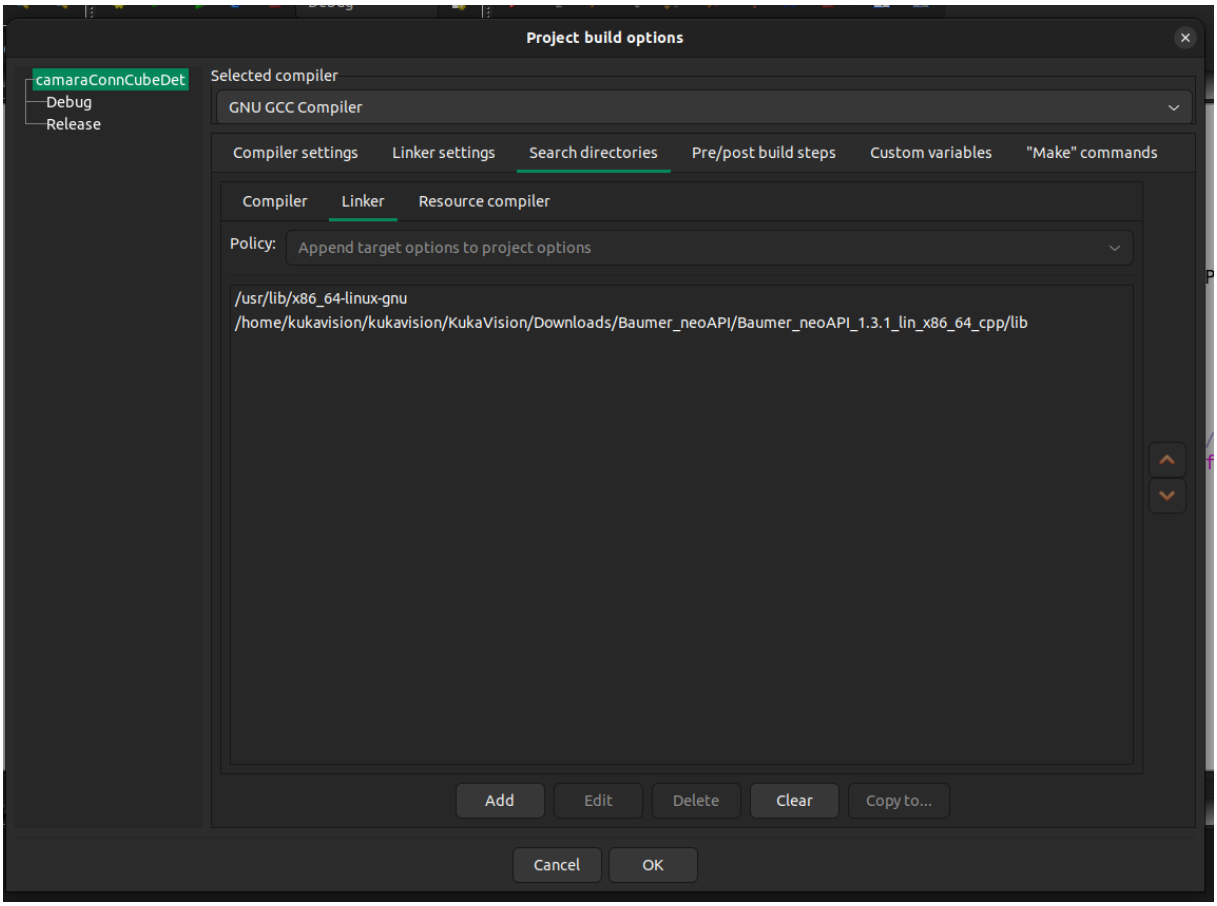
Start with the images you took. Go to **Project->Set program's arguments.** Add the correct paths of both images (reference).



Next step, the compiler settings and libraries. Go to **Project->Build options** and set it up like in the following images.

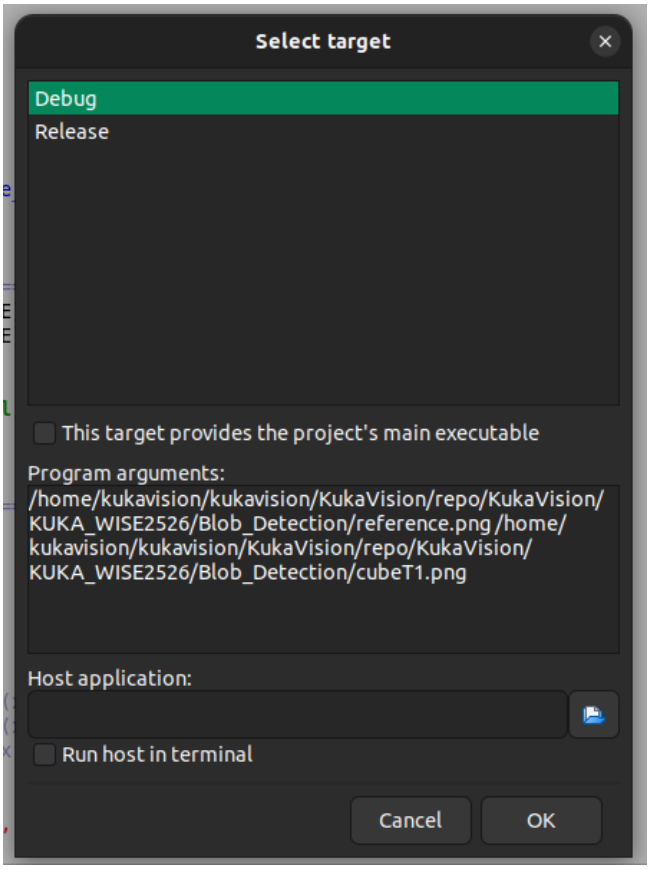



Project build options
camaraConnCubeDet
Debug
Release
Selected compiler
GNU GCC Compiler
Compiler settings
Linker settings
Search directories
Pre/post build steps
Custom variables
"Make" commands
Compiler
Linker
Resource compiler
Policy:
Append target options to project options
/usr/include/opencv4
/home/kukavision/kukavision/KukaVision/Downloads/Baumer_neoAPI/Baumer_neoAPI_1.3.1_lin_x86_64_cpp/include
Add
Edit
Delete
Clear
Copy to...
Cancel
OK

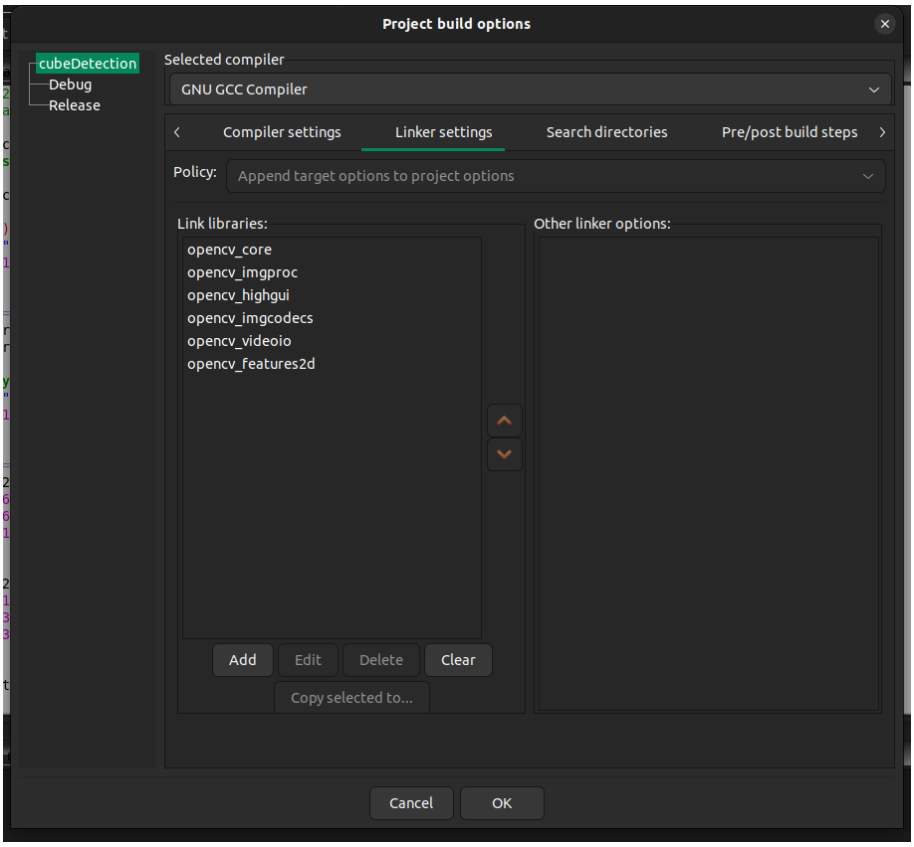

Project build options
camaraConnCubeDet
Debug
Release
Selected compiler
GNU GCC Compiler
Compiler settings
Linker settings
Search directories
Pre/post build steps
Custom variables
"Make" commands
Compiler
Linker
Resource compiler
Policy:
Append target options to project options
/usr/lib/x86_64-linux-gnu
/home/kukavision/kukavision/KukaVision/Downloads/Baumer_neoAPI/Baumer_neoAPI_1.3.1_lin_x86_64_cpp/lib
Add
Edit
Delete
Clear
Copy to...
Cancel
OK

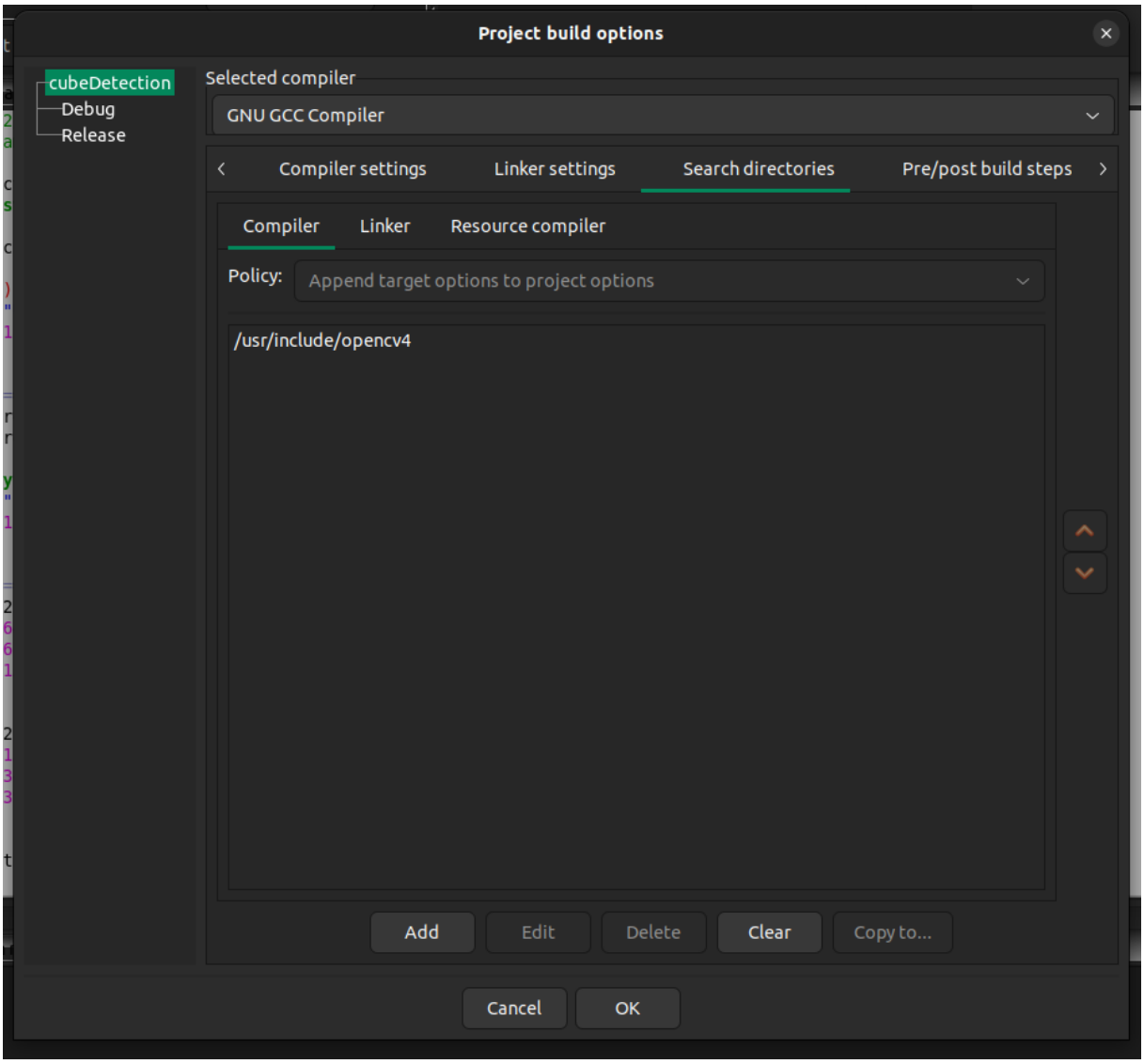
cubeDetection:

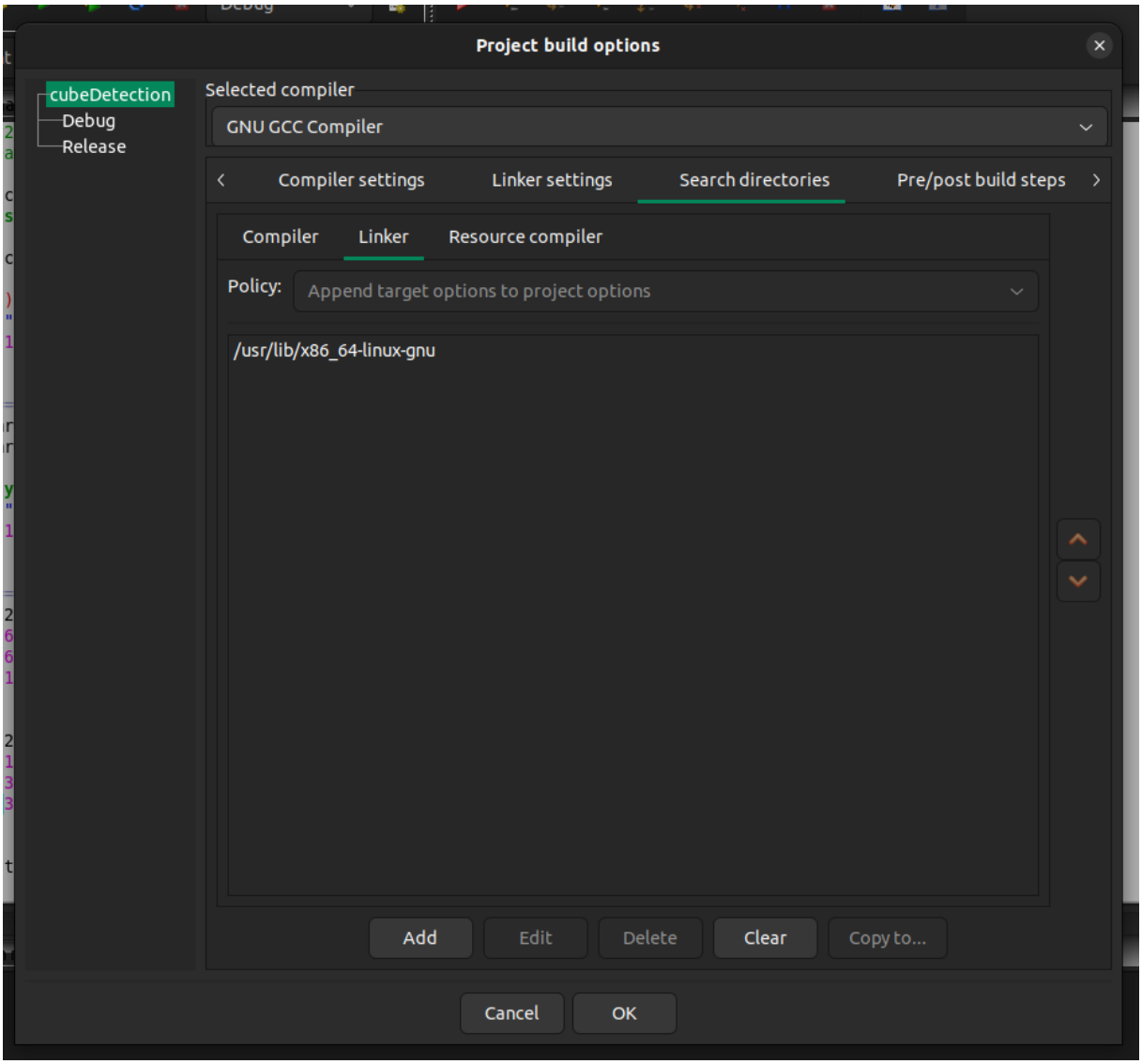
Go to **Project->Set program's arguments.** Add the correct path of the image without the cube (cubeT1 and reference).



Adjust now the compiler settings and libraries. Go to **Project->Build options** and set it up like in the following images.


Project build options
cubeDetection
Debug
Release
Selected compiler
GNU GCC Compiler
Compiler settings
Linker settings
Search directories
Pre/post build steps
Policy:
Append target options to project options
Link libraries:
opencv_core
opencv_imgproc
opencv_highgui
opencv_imgcodecs
opencv_videoio
opencv_features2d
Other linker options:
Add
Edit
Delete
Clear
Copy selected to...
Cancel
OK

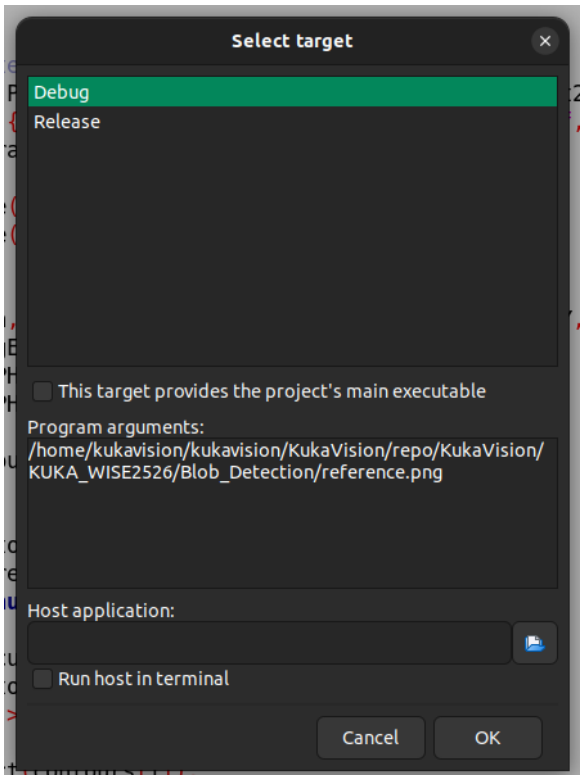

Project build options
cubeDetection
Debug
Release
Selected compiler
GNU GCC Compiler
Compiler settings
Linker settings
Search directories
Pre/post build steps
Compiler
Linker
Resource compiler
Policy:
Append target options to project options
/usr/include/opencv4
Add
Edit
Delete
Clear
Copy to...
Cancel
OK



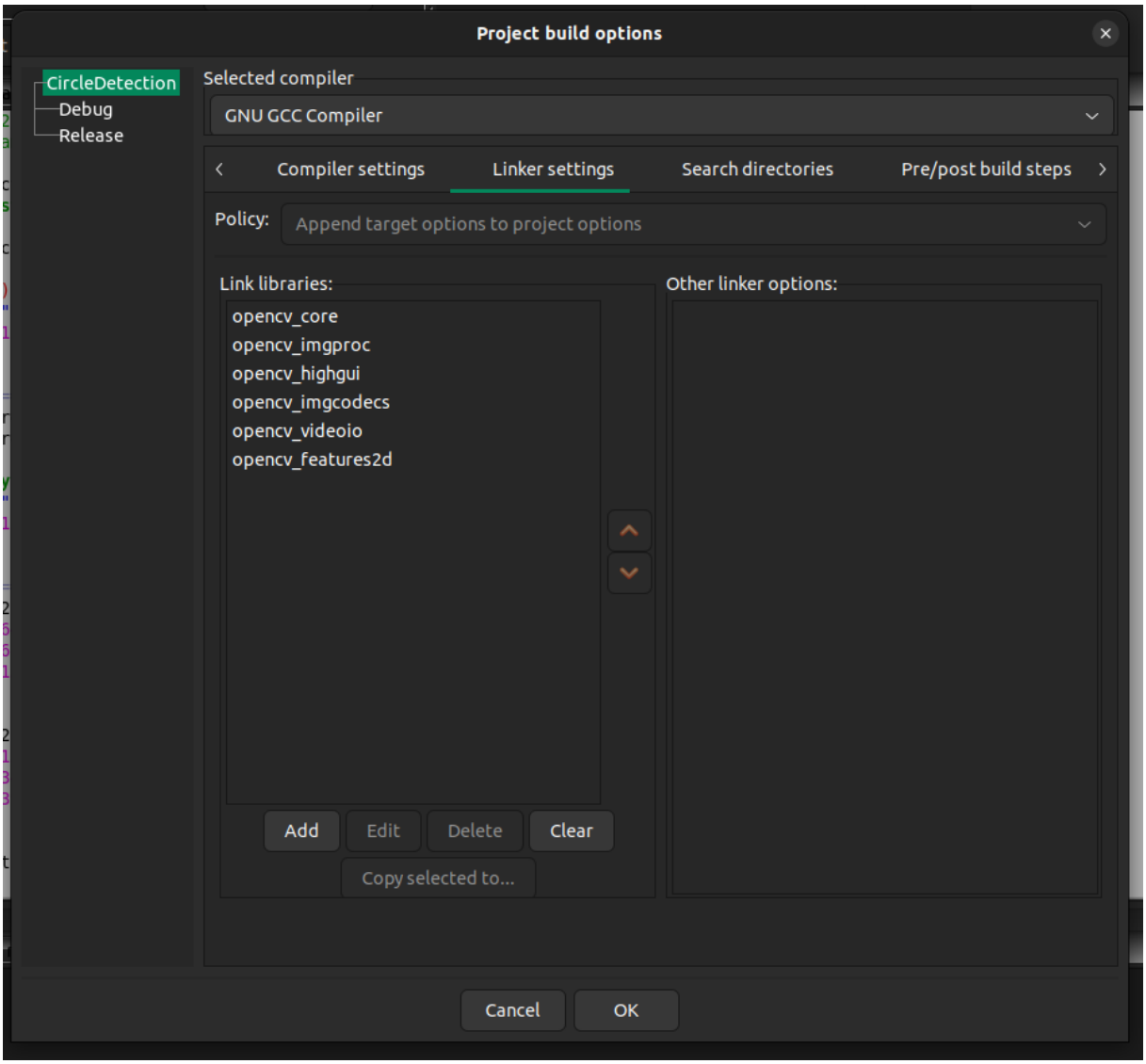
Place one cube on the workplace and run the code. On the screen you should see the outlined cube. This means, the camera is detecting the cube. If it is not detecting, try to take another picture of the workplace, or adjust the light conditions by turning on all lights in the room and closing the shutters and take picture than.

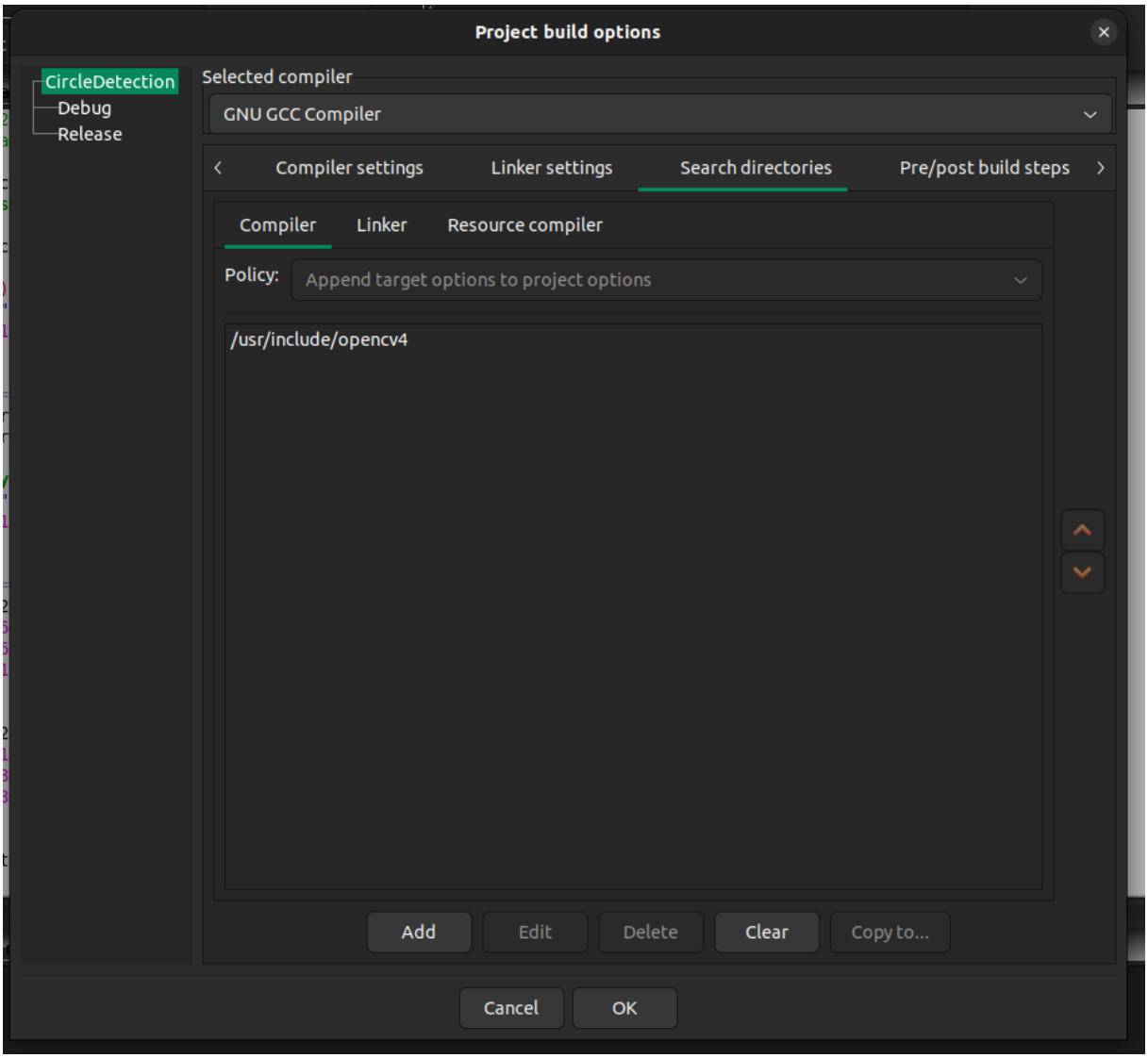
<u>circleDetection:</u>

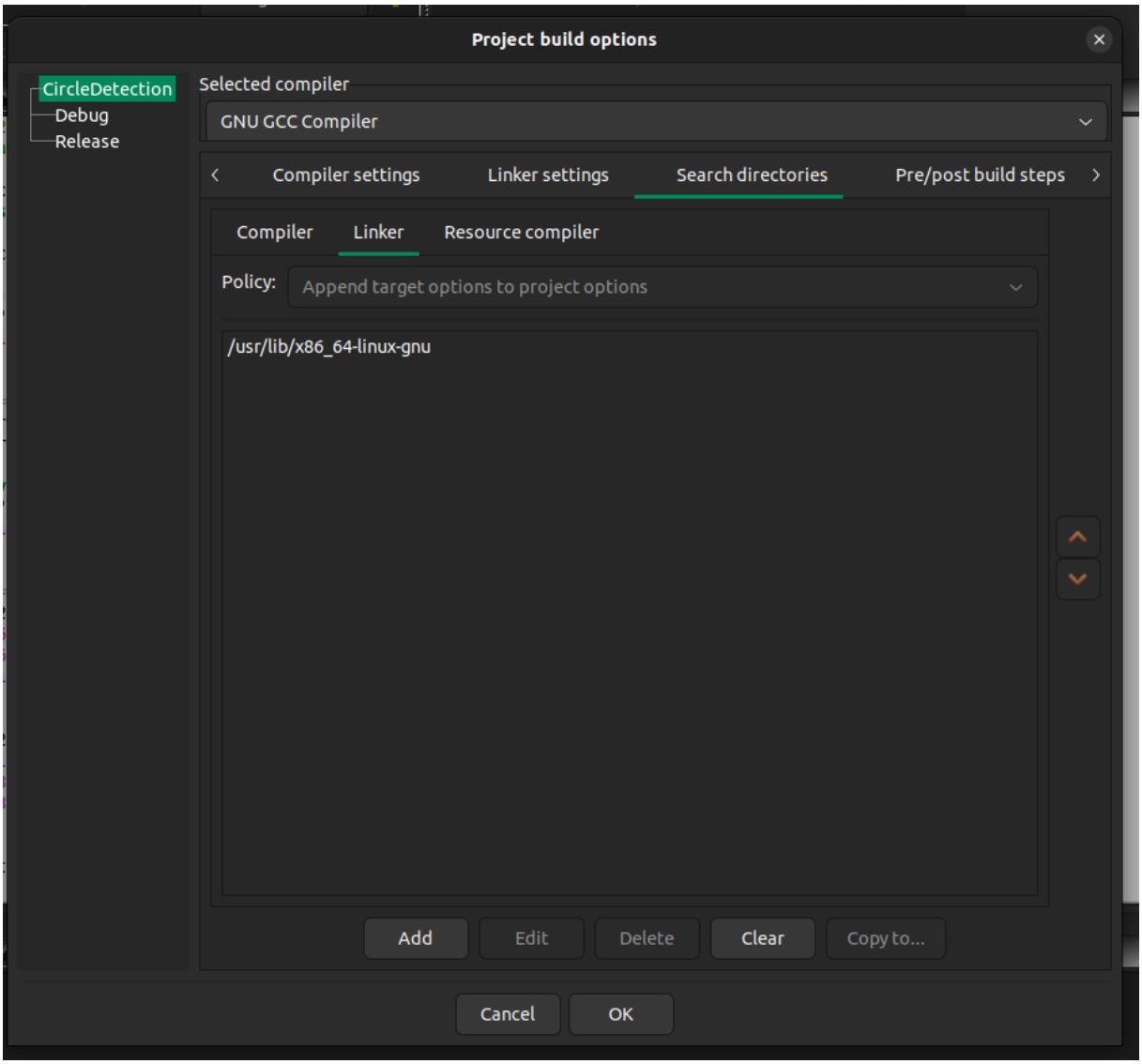
Go to **Project->Set program's arguments.** Add the correct path of the image without the cube (reference).



Adjust now the compiler settings and libraries. Go to **Project->Build options** and set it up like in the following images.


Project build options
CircleDetection
Debug
Release
Selected compiler
GNU GCC Compiler
Compiler settings
Linker settings
Search directories
Pre/post build steps
Policy:
Append target options to project options
Link libraries:
opencv_core
opencv_imgproc
opencv_highgui
opencv_imgcodecs
opencv_videoio
opencv_features2d
Other linker options:
Add
Edit
Delete
Clear
Copy selected to...
Cancel
OK


Project build options
CircleDetection
Debug
Release
Selected compiler
GNU GCC Compiler
Compiler settings
Linker settings
Search directories
Pre/post build steps
Compiler
Linker
Resource compiler
Policy:
Append target options to project options
/usr/include/opencv4
Add
Edit
Delete
Clear
Copy to...
Cancel
OK



The workplace should be empty. Run the code. The three black circles in the workplace must be detected. Each circle has its own pixel coordinate. Write them down. You need them in the main code.

Now write the pixel coordinates in the intended place in camaraConnCubeDet (line 123). If the camera was moved or the workspace changed its position even if it is very little changes, detect the coordinates again. The fourth circle is at the corner of the workspace without a circle. Maybe you should print a workspace which have four circles.

```
123  vector<Point2f> camPts = { Point2f(646, 248), Point2f(617, 770), Point2f(1395, 815), Point2f(1441, 286)};
124  vector<Point2f> robotPts = { Point2f(688.00f, 164.12f), Point2f(688.00f, 356.29f), Point2f(396.70f, 356.29f), Point2f(396.70f, 161.66f)};
```

Start the robot. Move the robot manually to the three black circles and write down the coordinates. The top of the left gripper is in this case the Tool Centre Point. Every robot coordinate must be detected in relation to it. So, move the top of the left gripper to the middle of the circle. Select the robot icon on the top left of the smartpad, go to Anzeigen->Istzustand

Add them in the main code here (line 124).

```
123  vector<Point2f> camPts = { Point2f(646, 248), Point2f(617, 770), Point2f(1395, 815), Point2f(1441, 286)};
124  vector<Point2f> robotPts = { Point2f(688.00f, 164.12f), Point2f(688.00f, 356.29f), Point2f(396.70f, 356.29f), Point2f(396.70f, 161.66f)};
```

On the smartpad go to R1\Program\movehome.src. Activate the program (“Anwählen”).

On the PC run the code camaraConnCubeDet. The terminal opens. Enter 1 for detection. An image of the workplace is to be seen. On the smartpad press on play one time, release, and hold it for the next steps. You get the message WAIT FOR FLAG 2. Change to the Terminal and enter 2. Now the robot should move to the cube, pick it up and place it on the rail.